

SISTEMA DI PARSING WEB E COSTRUZIONE DI GOLD STANDARD PER DOMINI SPECIFICI

Marco Grigoli, Daniele Antonacci, Alessio Colicci

1 Obiettivo del Progetto

L'obiettivo del progetto è sviluppare un sistema per l'estrazione e la pulizia di contenuti testuali da pagine web, eliminando elementi non rilevanti. Inoltre, il sistema consente la costruzione e il confronto con un gold standard per valutare la qualità del parsing.

2 Organizzazione del Codice

Il progetto è organizzato nei seguenti moduli:

Backend:

- **crawler.py** — implementa due metodi per parsare pagine: uno dinamico con l'url ed uno statico con l'html. In entrambi i metodi viene selezionato il CrawlerRunConfig specifico per il dominio di interesse.
- **server.py** — implementa la logica di vari endpoint FastAPI:
 - GET /parse: riceve un URL in input, esegue il parsing della pagina e restituisce un oggetto contenente l'URL parsato, il dominio, il titolo, l'HTML grezzo e il testo estratto in formato Markdown.
 - POST /parse: effettua parsing sull'HTML dato in input e restituisce un oggetto identico a quello di get /parse.
 - GET /domains: restituisce la lista dei domini supportati.
 - GET /gold_standard: restituisce il gold standard dell'URL, accedendo alla tabella gold_standard e web_resources del database, restituendo in output un oggetto contenente il gold_text.
 - GET /full_gold_standard: restituisce tutto il GS del dominio richiesto, controllando che il dominio sia supportato.
 - GET /full_gs_eval: prende in input un dominio tra quelli supportati e calcola la media delle valutazioni di POST /evaluate e di POST /evaluate_judge delle entry del Gold Standard per il dominio dato.
 - POST /evaluate: chiama dal modulo evaluator.py i metodi necessari a pulire dal markdown il parsed_text ottenuto come output dalla GET o POST /parse e il gold_text recuperato dal GS.json del dominio per calcolare le metriche richieste. Se uno tra parsed_text o gold_text è vuoto, restituisce metriche nulle.
 - POST /evaluate_judge: come POST /evaluate, riceve in input il parsed_text, pulendolo con i metodi chiamati dal modulo evaluator.py, e il gold_text e li invia tramite una richiesta HTTP al server Ollama locale. Il modello LLM utilizzato è llama3.2:3b, istruito tramite un prompt strutturato a dare un voto intero da 1 a 5 ed una risposta in formato JSON contenente lo score ed un feedback. Se parsed_text o gold_text sono vuoti, restituisce score nullo.
 - GET /gold_standard_urls: restituisce la lista degli URL presenti nel Gold Standard per un dominio specifico, interrogando il database tramite una join tra web_resources e gold_standard.
 - POST /add_web_resource: aggiunge una nuova risorsa web nella tabella web_resources del database, estraendo automaticamente dominio e titolo dall'HTML fornito in input tramite

BeautifulSoup.

- POST /add_gold_standard: aggiunge una nuova entry nella tabella gold_standard, richiedendo che la corrispondente web_resource esista già nel database. In caso contrario viene restituito un errore di integrità referenziale.
- DELETE /web_resource: rimuove una risorsa web dalla tabella web_resources; per effetto della foreign key con cascade, viene eliminata automaticamente anche l'eventuale entry associata in gold_standard.
- DELETE /gold_standard: rimuove solo l'entry dalla tabella gold_standard, lasciando intatta la risorsa web associata. Restituisce errore se l'URL non è presente nel GS.
- GET /db_stats: restituisce statistiche aggregate per dominio calcolate dai dati presenti nel database: numero di entry in web_resources e gold_standard, metriche medie di valutazione dalla tabella evaluations e score medio dalla tabella llm_judgments.
- GET /db_schema: interroga information_schema di MariaDB e restituisce lo schema reale del database, specificando per ogni colonna il tipo, la chiave primaria e l'eventuale chiave esterna.
- GET /status: verifica la raggiungibilità dei tre componenti del sistema (backend, database, Ollama) e restituisce per ciascuno il valore ok o error.

Tutti i metodi implementano una logica di gestione errori, in particolare dominio non supportato per il parsing o URL irraggiungibile.

- **evaluator.py** - questo modulo si occupa di tokenizzare il testo parsato precedentemente ripulito dei simboli del markdown ed il gold text facendo uso di un multi-insieme implementato attraverso Counter. Calcola poi le metriche di valutazione standard assicurandosi di coprire di casi di divisione per 0.
- **remove_markdown.py**: - implementazione del prof per la rimozione del markdown. In server avviene la conversione nei caratteri unicode corrispondenti tramite `html.unescape()`.

Frontend:

- **frontend.py** — implementa l'interfaccia web dell'applicazione utilizzando FastAPI e le librerie requests e Jinja2. Implementa la pagina home e le pagine a cui accediamo attraverso di essa: una per osservare le valutazioni di uno specifico url, una per poter aggiungere un nuovo URL nel database appartenente ad uno dei domini supportati o per eliminare una entry del Gold Standard da un dominio, ed una per visualizzare delle statistiche del database.
- **index.html** — rappresenta l'interfaccia utente della sezione Parser & Evaluation ed è realizzato utilizzando Jinja2. Contiene un form che permette all'utente di inserire un URL e un menu a tendina con una lista di gold standard disponibili per il dominio selezionato. Il template gestisce la visualizzazione dinamica dei dati ricevuti dal backend, mostrando il contenuto parsato e, quando disponibile, il relativo gold standard con le metriche di valutazione token_level_eval e il giudizio del modello LLM tramite POST /evaluate_judge, comprensivo di score e feedback testuale. Utilizza costrutti condizionali per mostrare i risultati solo quando presenti e per gestire eventuali messaggi di errore.
- **home.html** — pagina principale dell'interfaccia, mostra lo stato dei componenti del sistema (backend, database, Ollama) recuperato tramite GET /status, la lista dei domini supportati ottenuta da GET /domains, e i collegamenti alle tre sezioni principali dell'applicazione.
- **gs_page.html** — interfaccia per la gestione del Gold Standard. Permette di selezionare un dominio e inserire un URL per scaricare l'HTML grezzo tramite il parsing live di POST /parse; l'HTML viene mostrato in un'area di testo per consentire l'estrazione manuale del testo informativo. Una volta inserito il gold text, il submit salva la risorsa nel database invocando POST /add_web_resource e POST /add_gold_standard in sequenza. La pagina mostra inoltre la lista degli URL già presenti nel Gold Standard per il dominio selezionato, con la possibilità di eliminare singole entry tramite DELETE /gold_standard.

- **stats_page.html** — La pagina mostra una panoramica dei dati presenti nel database per ciascun dominio. Mostra per ciascun dominio il numero di entry nelle tabelle `web_resources` e `gold_standard`, le metriche medie `token_level_eval` (precision, recall, F1) e il judge score medio ottenuto dal modello LLM, tutti recuperati tramite `GET /db_stats`.

Gs Data:

La sezione `gsdata` contiene tutte le informazioni principali dei domini supportati: ogni dominio è suddiviso in `N` directories, quanti gli `N` URL supportati. All'interno di ogni directory è presente un parser ad hoc per URL, che insieme al testo copiato a mano in `gsX` (`X` da 1 a `N`), sono forniti a `json_generator.py` che automatizza il processo di creazione del gold di un URL. Il modulo `GS_generator.py` invece automatizza la creazione di gold standard per dominio, accorpando quelli di ogni suo URL.

3 Valutazione del Parser

Le metriche utilizzate sono quelle standard. Di seguito le tabelle con le valutazioni per ogni dominio.

Dominio	F1-score
en.wikipedia.org	0.8704
www.ecb.europa.eu	0.8757
www.tandfonline.com	0.9452
apps.apple.com	0.9962

4 Valutazione del Judge

Le metriche utilizzate sono quelle standard. Il modello utilizzato è `llama3.2:3b`, ed il prompt è focalizzato nell'avere un contesto solido ed istruzioni ottimali con una specifica del formato di output che si richiede all'LLM. Di seguito le tabelle con le valutazioni del Judge per ogni dominio limitando i caratteri degli input a 4000.

Dominio	Score
en.wikipedia.org	5
www.ecb.europa.eu	5
www.tandfonline.com	5
apps.apple.com	5

5 Organizzazione del Database

MariaDB, che gira in un container dedicato e comunicante con gli altri, gestisce il database di 4 tabelle composto dalle due obbligatorie:

- **web_resources**: con primary key `url`, contiene il dominio, il titolo, l'HTML grezzo della pagina e il timestamp di creazione. È la tabella principale da cui dipendono tutte le altre.
- **gold_standard**: con primary key `url` che è anche foreign key verso `web_resources.url`, contiene il testo di riferimento "corretto" contro cui confrontare il parsing e il timestamp di creazione.

E le 2 aggiuntive:

- **evaluations**: con primary key `url` (foreign key verso `web_resources.url`), contiene le metriche di valutazione token-level calcolate dal metodo `evaluate`: precision, recall e F1-score, più il timestamp.
- **llm_judgments**: con primary key `url` (foreign key verso `web_resources.url`), contiene le valutazioni del metodo `evaluate_judge`: lo score intero (1-5) assegnato dal modello LLM, il feedback testuale (`verdict`) e il timestamp.

Tutte le tabelle secondarie hanno una relazione 1:1 opzionale con `web_resources`. Inoltre, è stata impostata la cancellazione in cascata (`ON DELETE CASCADE`) sulle foreign key per garantire l'integrità referenziale dei dati e l'eliminazione automatica delle entry associate.

6 Note Aggiuntive

I domini `www.tandfonline.com` e `apps.apple.com` rischiano il fallimento durante l'esecuzione del crawler sulla pagina web (parsing live) in quanti implementano politiche anti-bot. Nel server di backend si fa uso di parsing direttamente sugli HTML, quindi non si incappa in questo errore. Tuttavia, nella pagina di creazione del Gold Standard del frontend è necessario il parsing live, per cui la possibilità per quei due domini del fallimento del parsing non dipende dall'implementazione degli endpoint. Questa instabilità è di rado presente anche nel dominio EBC (Banca Europea), ma riteniamo corretto segnalarlo.